

lab7-dependency-parsing

December 2, 2025

Lab 7: Thực hành chuyên sâu về Phân tích cú pháp phụ thuộc (Dependency Parsing)

Giang Nguyen Thi - 22001254

2025-12-02

Mục tiêu

Sau buổi thực hành này, bạn sẽ có khả năng:

1. Sử dụng thư viện spaCy để thực hiện phân tích cú pháp phụ thuộc cho một câu.
2. Trực quan hóa cây phụ thuộc để hiểu rõ cấu trúc câu.
3. Truy cập và duyệt (traverse) cây phụ thuộc bằng chương trình.
4. Trích xuất thông tin có ý nghĩa từ các quan hệ phụ thuộc (ví dụ: chủ ngữ, tân ngữ, bổ ngữ).

Yêu cầu

- Python 3.6 trở lên
- Thư viện spaCy và mô hình ngôn ngữ tiếng Anh

Hướng dẫn chạy code

Chạy từng cell trong .ipynb này

0.1 Phần 1: Giới thiệu và Cài đặt

```
[ ]: !pip install -U spacy
```

```
[ ]: !python -m spacy download en_core_web_md
```

0.2 Phần 2: Phân tích câu và Trực quan hóa

0.2.1 2.1. Tải mô hình và phân tích câu

```
[10]: import spacy
from spacy import displacy
# Tải mô hình tiếng Anh đã cài đặt
# Sử dụng en_core_web_md vì nó chứa các vector từ và cây cú pháp đầy đủ
nlp = spacy.load("en_core_web_md")
# Câu ví dụ
text = "The quick brown fox jumps over the lazy dog."
```

```
# Phân tích câu với pipeline của spaCy
doc = nlp(text)
```

0.2.2 2.2. Trực quan hóa cây phụ thuộc

displaCy có thể khởi chạy một web server nhỏ để hiển thị kết quả.

```
[ ]: # Tùy chọn để hiển thị trong trình duyệt
# options = {"compact": True, "color": "blue", "font": "Source Sans Pro"}
# Khởi chạy server tại http://127.0.0.1:5000
# Bạn có thể truy cập địa chỉ này trên trình duyệt để xem cây phụ thuộc
# Nhấn Ctrl+C trong terminal để dừng server
displacy.serve(doc, style="dep")
```

```
c:\Users\DELL\AppData\Local\Programs\Python\Python312\Lib\site-
packages\spacy\displacy\__init__.py:108: UserWarning: [W011] It looks like
you're calling displacy.serve from within a Jupyter notebook or a similar
environment. This likely means you're already running a local web server, so
there's no need to make displaCy start another one. Instead, you should be able
to replace displacy.serve with displacy.render to show the visualization.
```

```
warnings.warn(Warnings.W011)
```

```
<IPython.core.display.HTML object>
```

Using the 'dep' visualizer

Serving on http://0.0.0.0:5000 ...

```
127.0.0.1 - - [02/Dec/2025 14:44:26] "GET / HTTP/1.1" 200 7538
```

```
127.0.0.1 - - [02/Dec/2025 14:44:27] "GET /favicon.ico HTTP/1.1" 200 7538
```

Câu hỏi

Câu 1. Từ nào là gốc (ROOT) của câu?

=> ROOT = “jumps”.

Trong phân tích cú pháp phụ thuộc, động từ chính của câu thường là ROOT. Trong hình, mũi tên lớn nhất trở xuống từ “jumps” thể hiện nó là trung tâm (head) của toàn bộ câu.

Câu 2. “jumps” có những từ phụ thuộc (dependent) nào? Các quan hệ đó là gì?

=> Từ “jumps” là động từ chính (ROOT) và có hai từ phụ thuộc trực tiếp:

Từ phụ thuộc	Loại từ	Nhãn quan hệ (dep_)	Ý nghĩa
fox	NOUN	nsubj	Chủ ngữ của động từ “jumps”
over	ADP	prep	Giới từ phụ thuộc vào động từ

Tóm lại: - “fox” là chủ thể thực hiện hành động. - “over” gắn với động từ và dẫn đến tân ngữ “dog” qua quan hệ pobj.

Câu 3: “fox” là head của những từ nào?

Từ “fox” là danh từ và là head của ba từ sau:

Từ phụ thuộc	Loại từ	Nhãn quan hệ (dep_)	Ý nghĩa
The	DET	det	Mạo từ của “fox”
quick	ADJ	amod	Tính từ bổ nghĩa
brown	ADJ	amod	Tính từ bổ nghĩa

Cụm danh từ đầy đủ: “The quick brown fox”.

0.3 Phần 3: Truy cập các thành phần trong cây phụ thuộc

Trực quan hóa rất hữu ích, nhưng sức mạnh thực sự đến từ việc truy cập cây phụ thuộc theo chương trình. Mỗi Token trong đối tượng Doc của spaCy chứa đầy đủ thông tin về vị trí của nó trong cây.

```
[4]: # Lấy một câu khác để phân tích
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)

# In ra thông tin của từng token
print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | _
↳ {'CHILDREN'}")
print("-" * 70)

for token in doc:
    # Trích xuất các thuộc tính
    children = [child.text for child in token.children]
    print(f"{token.text:<12} | {token.dep_:<10} | {token.head.text:<12} | _
↳ {token.head.pos_:<8} | {children}")
```

TEXT	DEP	HEAD TEXT	HEAD POS	CHILDREN
Apple	nsubj	looking	VERB	[]
is	aux	looking	VERB	[]
looking	ROOT	looking	VERB	['Apple', 'is', 'at']
at	prep	looking	VERB	['buying']
buying	pcomp	at	ADP	['startup']
U.K.	compound	startup	NOUN	[]
startup	dobj	buying	VERB	['U.K.', 'for']
for	prep	startup	NOUN	['billion']
\$	quantmod	billion	NUM	[]
1	compound	billion	NUM	[]
billion	pobj	for	ADP	['\$', '1']

Giải thích các thuộc tính trong spaCy Token

- token.text: văn bản gốc của token trong câu.

- token.dep_: nhãn quan hệ phụ thuộc (dependency label) mô tả mối quan hệ của token này với head của nó trong cây cú pháp.
- token.head.text: văn bản của token head mà token hiện tại phụ thuộc vào.
- token.head.pos_: loại từ (Part-of-Speech tag) của token head.
- token.children: một iterator chứa danh sách các token con (dependents) trực tiếp của token hiện tại.

0.4 Phần 4: Duyệt cây phụ thuộc để trích xuất thông tin

0.4.1 4.1. Bài toán: Tìm chủ ngữ và tân ngữ của một động từ

```
[6]: text = "The cat chased the mouse and the dog watched them."
doc = nlp(text)

for token in doc:
    # Chỉ tìm các động từ
    if token.pos_ == "VERB":
        verb = token.text
        subject = ""
        obj = ""

        # Tìm chủ ngữ (nsubj) và tân ngữ (dobj)
        for child in token.children:
            if child.dep_ == "nsubj":
                subject = child.text
            if child.dep_ == "dobj":
                obj = child.text

        # Nếu đầy đủ chủ ngữ và tân ngữ thì in ra
        if subject and obj:
            print(f"Found Triplet: ({subject}, {verb}, {obj})")
```

Found Triplet: (cat, chased, mouse)

Found Triplet: (dog, watched, them)

0.4.2 4.2. Bài toán: Tìm các tính từ bổ nghĩa cho một danh từ

```
[9]: text = "The big, fluffy white cat is sleeping on the warm mat."
doc = nlp(text)

for token in doc:
    # Chỉ tìm các danh từ
    if token.pos_ == "NOUN":
        adjectives = []

        # Tìm các tính từ bổ nghĩa (amod)
```

```

for child in token.children:
    if child.dep_ == "amod":
        adjectives.append(child.text)

if adjectives:
    print(f"Danh từ '{token.text}' được bổ nghĩa bởi các tính từ:␣
↪{adjectives}")

```

Danh từ 'cat' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']
Danh từ 'mat' được bổ nghĩa bởi các tính từ: ['warm']

0.5 Phần 5: Bài tập tự luyện

0.5.1 Bài 1: Tìm động từ chính của câu

```

[15]: def find_main_verb(doc):
    for token in doc:
        if token.dep_ == "ROOT" and token.pos_ == "VERB":
            return token
    return None

```

```

[16]: text1 = "The quick brown fox jumps over the lazy dog."
text2 = "The big, fluffy white cat is sleeping on the warm mat."

doc1 = nlp(text1)
doc2 = nlp(text2)

main_verb1 = find_main_verb(doc1)
main_verb2 = find_main_verb(doc2)

print("Động từ chính của câu 1 là:", main_verb1.text)
print("Động từ chính của câu 2 là:", main_verb2.text)

```

Động từ chính của câu 1 là: jumps
Động từ chính của câu 2 là: sleeping

Giải thích kết quả

Hàm `find_main_verb(doc)` duyệt qua tất cả các token trong câu và trả về token có nhãn phụ thuộc `ROOT`. Trong phân tích cú pháp phụ thuộc của spaCy, `ROOT` là động từ chính của câu, đại diện cho hành động trung tâm mà các thành phần khác hướng tới.

Khi áp dụng hàm cho hai câu:

1. Câu 1:
The quick brown fox jumps over the lazy dog.
Kết quả trả về: jumps
Đây là động từ mô tả hành động chính của chủ ngữ fox.
2. Câu 2:

The big, fluffy white cat is sleeping on the warm mat.

Kết quả trả về: sleeping

Đây là động từ ở dạng tiếp diễn (present participle), mô tả hành động mà cat đang thực hiện.

Kết quả thu được phù hợp với cấu trúc ngữ pháp của hai câu và đúng với nguyên tắc: động từ mang nhãn ROOT chính là động từ chính của câu.

0.5.2 Bài 2: Trích xuất các cụm danh từ (Noun Chunks)

Trong spaCy, thuộc tính `doc.noun_chunks` giúp trích xuất cụm danh từ tự động.

Trong bài này, tự viết lại bằng cách:

- Tìm các danh từ (NOUN)
- Tìm các từ bổ nghĩa cho nó:
 - `det` = từ hạn định (the, a...)
 - `amod` = tính từ (big, white...)
 - `compound` = danh từ bổ sung (U.K., data, car door...)
- Gom lại các từ đó theo thứ tự trong câu => tạo thành cụm danh từ.

```
[21]: def extract_noun_chunks(doc):  
    noun_chunks = []  
  
    for token in doc:  
        # Nếu token là danh từ  
        if token.pos_ == "NOUN":  
            modifiers = []  
  
            # Lấy các từ bổ nghĩa: det, amod, compound  
            for child in token.children:  
                if child.dep_ in ("det", "amod", "compound"):  
                    modifiers.append(child)  
  
            # Thêm chính danh từ vào  
            modifiers.append(token)  
  
            # Sắp xếp theo thứ tự trong câu  
            modifiers = sorted(modifiers, key=lambda w: w.i)  
  
            # Tạo cụm danh từ  
            chunk_text = " ".join([w.text for w in modifiers])  
            noun_chunks.append(chunk_text)  
  
    return noun_chunks
```

```
[22]: text = "The big, fluffy white cat is sleeping on the warm mat."  
doc = nlp(text)
```

```
chunks = extract_noun_chunks(doc)
print(chunks)
```

```
['The big fluffy white cat', 'the warm mat']
```

Giải thích kết quả

Khi chạy hàm `extract_noun_chunks(doc)` với câu:

The big, fluffy white cat is sleeping on the warm mat.

Hàm trả về danh sách:

```
['The big fluffy white cat', 'the warm mat']
```

Giải thích

1. Cụm danh từ: The big fluffy white cat
 - Danh từ head: cat
 - Các từ bổ nghĩa của cat:
 - The => det (từ hạn định)
 - big => amod (tính từ)
 - fluffy => amod (tính từ)
 - white => amod (tính từ)
 - Khi sắp xếp theo thứ tự xuất hiện trong câu, ta thu được cụm danh từ:
The big fluffy white cat
2. Cụm danh từ: the warm mat
 - Danh từ head: mat
 - Các từ bổ nghĩa:
 - the => det
 - warm => amod
 - Ghép lại theo thứ tự trong câu thành cụm:
the warm mat

Kết luận

Hàm đã xác định đúng các danh từ và các từ bổ nghĩa của chúng dựa trên quan hệ phụ thuộc (det, amod, compound).

Kết quả trích xuất phù hợp với cấu trúc ngữ pháp và tương đương với những gì `doc.noun_chunks` của spaCy sẽ tạo ra.

0.5.3 Bài 3: Tìm đường đi ngắn nhất trong cây

```
[8]: def get_path_to_root(token):
    path = []
    current = token

    while True:
        path.append(current)
```

```

    if current.dep_ == "ROOT":
        break
    current = current.head

return path

```

```

[14]: text = "The quick brown fox jumps over the lazy dog."
doc = nlp(text)

# chọn token bất kỳ, ví dụ: 'fox'
token = doc[1]    # fox

path = get_path_to_root(token)

print("Token:", token.text)
print("Đường đi từ token đến ROOT:")
for t in path:
    print(" -", t.text, "(", t.dep_, ")")

```

Token: quick

Đường đi từ token đến ROOT:

```

- quick ( amod )
- fox ( nsubj )
- jumps ( ROOT )

```

Giải thích

1. “quick” mang quan hệ phụ thuộc **amod**, tức là một tính từ bổ nghĩa cho danh từ “fox”.
2. “fox” mang quan hệ **nsubj**, nghĩa là chủ ngữ của động từ “jumps”.
3. “jumps” là động từ chính của câu, được gán nhãn **ROOT**.

Do đó, đường đi từ “quick” lên đến ROOT là:

quick => foxc jumps

Kết quả này phù hợp với cấu trúc dependency tree của câu và cho thấy hàm `get_path_to_root` hoạt động đúng.

0.6 Khó khăn và giải pháp khi thực hiện Lab

Trong quá trình thực hiện lab phân tích cú pháp phụ thuộc với spaCy, một số khó khăn gặp phải cùng với cách xử lý như sau:

1. Hiểu cấu trúc Dependency và các nhãn quan hệ
Ban đầu khó hình dung cách các token được liên kết trong cấu trúc head-dependent, nhất là các nhãn như **nsubj**, **dobj**, **amod**, **pobj**, **aux**.
Giải pháp: sử dụng trực quan hóa bằng displaCy để dễ dàng quan sát cây phụ thuộc.
2. Phân biệt giữa POS tag và Dependency label
Dễ nhầm lẫn giữa loại từ (POS) và nhãn phụ thuộc (dep).
Giải pháp: in bảng gồm TEXT - POS - DEP - HEAD để quan sát rõ vai trò của từng token.

3. Duyệt cây phụ thuộc bằng code

Việc thao tác với `token.children` và `token.head` lúc đầu hơi khó do chưa quen cấu trúc của spaCy.

Giải pháp: thử nghiệm với các câu đơn giản và in ra từng thuộc tính để hiểu cách spaCy tổ chức cây.

4. Viết hàm trích xuất noun chunks thủ công

Cần xác định đúng các quan hệ bổ nghĩa (`det`, `amod`, `compound`) và sắp xếp chúng theo thứ tự xuất hiện.

Giải pháp: xây dựng từng bước, kiểm tra từng token rồi hoàn thiện hàm.

5. Tìm đường đi lên ROOT

Ban đầu chưa rõ việc mỗi token chỉ có một head nên đường đi lên ROOT là duy nhất.

Giải pháp: theo dõi quá trình `token ==> head ==> head ==> ROOT` để hiểu rõ nguyên tắc.

Các giải pháp trên giúp hoàn thành lab một cách hiệu quả hơn và hiểu sâu hơn về dependency parsing trong spaCy.

0.7 Tài liệu tham khảo

1. spaCy Documentation - Dependency Parsing
<https://spacy.io/usage/linguistic-features#dependency-parse>
2. spaCy API Reference - Token attributes
<https://spacy.io/api/token>
3. spaCy Visualizer (displaCy)
<https://spacy.io/usage/visualizers>
4. Bài giảng và tài liệu hướng dẫn của lab trong môn học
5. Universal Dependencies (UD) - Giải thích nhãn phụ thuộc
<https://universaldependencies.org/u/dep/>